

STACK QNO:2

QUESTION:

Question description

Rajinikanth organized a technical round interview in an Animation company for the set of computer science candidates.

the task is to implement stack operations for two stacks and merge the stacks into one.

Get two sets of stack elements and reverse them, then merge them into one stack.

Rajinikanth has given the deadline of only 15 minutes to complete the problem.

Can you Help the candidates to complete the problem within the specified time limit?

Function Description

- a) push(): Adds the new item at the beginning of the linked list using the first pointer.
- b) pop(): Removes an item from the beginning using the first pointer.
- c) merge(): Links the first pointer second stack as next of the last pointer of the first list.

Constraints

$0 < n, m < N$

$1 < arr[i] < 1000$

Input Format:

First-line indicates n & m, where n is the number of elements to be pushed into the stack and m is the number of pop operations that need to be performed

The next line indicates the n number stack elements

Output Format:

First-line indicates the top of the element of the stack

the second line indicates the top of the element after the pop operation

✓ Logical Test Cases

Test Case 1	Test Case 2
INPUT (STDIN) 5 8 9 8 7 6 5 5 6 7 8 3 11 2 3 EXPECTED OUTPUT 5 6 7 8 9 3 2 11 3 8 7 6 5	INPUT (STDIN) 10 4 91 18 17 16 15 15 16 17 18 13 11 12 13 14 EXPECTED OUTPUT 13 18 17 16 15 15 16 17 18 91 14 13 12 11

✓ Mandatory Test Cases

Test Case 1	Test Case 2	Test Case 3
KEYWORD void push(int data, mystack* ms)	KEYWORD int pop(mystack* ms)	KEYWORD void merge(mystack* ms1, mystack* ms2)

✓ Complexity Test Cases

Test Case 1	Test Case 2	Test Case 3
CYCLOMATIC COMPLEXITY 1	TOKEN COUNT 435	NLOC 90

Source Code:

```
#include <stdio.h>
#include <stdlib.h>
typedef struct Node {
    int data;
    struct Node* next;
} Node;

typedef struct mystack {
    Node* head;
} mystack;

void push(int data, mystack* ms) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = data;
    newNode->next = ms->head;
    ms->head = newNode;
}

int pop(mystack* ms) {
    if (ms->head == NULL) {
        return -1;
    }
    Node* temp = ms->head;
    int data = temp->data;
    ms->head = ms->head->next;
    free(temp);
    return data;
}
```

```

void merge(mystack* ms1, mystack* ms2) {
    if (ms1->head == NULL) {
        ms1->head = ms2->head;
        return;
    }
    Node* temp = ms1->head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = ms2->head;
}

int main() {
    int n, m;
    if (scanf("%d %d", &n, &m) != 2) {
        return 0;
    }

    mystack ms1 = {NULL};
    mystack ms2 = {NULL};
    for (int i = 0; i < n; i++) {
        int val;
        scanf("%d", &val);
        push(val, &ms1);
    }

```

```

    for (int i = 0; i < m; i++) {
        int val;
        scanf("%d", &val);
        push(val, &ms2);
    }

    merge(&ms1, &ms2);

    Node* curr = ms1.head;
    int first = 1;
    while (curr != NULL) {
        if (!first) {
            printf(" ");
        }
        printf("%d", curr->data);
        first = 0;
        curr = curr->next;
    }
    printf("\n");
    return 0;
}

```

Output:

```

5 8
9 8 7 6 5
5 6 7 8 3 11 2 3
5 6 7 8 9 3 2 11 3 8 7 6 5

```

```
10 4
91 18 17 16 15 15 16 17 18 13
11 12 13 14
13 18 17 16 15 15 16 17 18 91 14 13 12 11
```